Deep Learning Guide using TensorFlow and PyTorch

Module 1: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        print(model.summary())


        import torch
        import torch.nn as nn

        class NeuralNetwork(nn.Module):
            def __init__(self):
                super().__init__()

                self.layer1 = nn.Linear(784, 256)
                self.relu = nn.ReLU()
                self.layer2 = nn.Linear(256, 10)

            def forward(self, x):
                x = self.relu(self.layer1(x))
                return self.layer2(x)
```

Module 2: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        print(model.summary())


        import torch
        import torch.nn as nn

        class NeuralNetwork(nn.Module):
            def __init__(self):
                super().__init__()

                self.layer1 = nn.Linear(784, 256)
                self.relu = nn.ReLU()
                self.layer2 = nn.Linear(256, 10)

            def forward(self, x):
                x = self.relu(self.layer1(x))
                return self.layer2(x)
```

Module 3: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        print(model.summary())
```

```
import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)
```

Module 4: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print(model.summary())
```

```
import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)
```

Module 5: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        print(model.summary())


        import torch
        import torch.nn as nn

        class NeuralNetwork(nn.Module):
            def __init__(self):
                super().__init__()

                self.layer1 = nn.Linear(784, 256)
                self.relu = nn.ReLU()
                self.layer2 = nn.Linear(256, 10)

            def forward(self, x):
                x = self.relu(self.layer1(x))
                return self.layer2(x)
```

Module 6: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
```

```
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print(model.summary())
```

```
import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)
```

Module 7: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print(model.summary())
```

```
import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)
```

Module 8: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(128, activation='relu'),
            Dense(64, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        print(model.summary())


        import torch
        import torch.nn as nn

        class NeuralNetwork(nn.Module):
            def __init__(self):
                super().__init__()

                self.layer1 = nn.Linear(784, 256)
                self.relu = nn.ReLU()
                self.layer2 = nn.Linear(256, 10)

            def forward(self, x):
                x = self.relu(self.layer1(x))
                return self.layer2(x)
```

Module 9: Deep Learning Systems

Deep Learning uses layered neural architectures to learn hierarchical representations from raw data. The increase in GPU power and large datasets enabled significant breakthroughs in computer vision and NLP.

Backpropagation computes gradients across layers using the chain rule. Optimizers such as Adam, RMSProp, and SGD are responsible for parameter updates.

Convolutional Neural Networks excel at spatial learning tasks, while Transformers dominate sequence modeling and language tasks.

```
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense
        model = Sequential([
```

```
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print(model.summary())


import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)
```